National University of Computer and Emerging Sciences
(FAST-NUCES)
PROJECT PROPOSAL

Group Members:

- Muhammad Adil Saeed – 24K-0705
- Harsh-Dembla-24K-0912
- Muhammad Haseem Samo – 24K-0666

Project Proposal: Restaurant Order Management Simulator

1. Introduction

In modern restaurants, efficient order management is essential to maintain fast service and customer satisfaction. Restaurants often receive multiple orders simultaneously, which must be processed efficiently by kitchen staff. This project proposes the development of a **Restaurant Order Management Simulator**, which models a multi-threaded restaurant kitchen environment.

In this system, waiter threads simulate customer order requests and place them into a shared order queue. Chef threads retrieve these orders from the queue and prepare them concurrently. The project demonstrates important operating system concepts such as **producer-consumer synchronization, thread coordination, and resource allocation**.

Mutex locks will protect the shared order queue to prevent race conditions when multiple threads access it. Semaphores will control the number of chefs working at the same time by limiting the available kitchen stations. This simulation reflects real-life restaurant order management and demonstrates how concurrency improves efficiency in real-world systems.

---

2. System Functionality

The system will simulate restaurant order processing using threads and synchronization techniques. The main features include:

- Waiter threads that generate and submit customer orders.
- A shared order queue where all incoming orders are stored.
- Mutex synchronization to protect the order queue from concurrent access issues.
- Chef threads that retrieve and prepare orders from the queue.
- Semaphore-controlled kitchen stations that limit how many chefs can work simultaneously.
- Order completion notification when a chef finishes preparing an order.

Additional advanced features include:

- Priority-based processing for VIP customer orders.
- Dynamic chef allocation depending on kitchen workload.
- Order cancellation and re-prioritization.
- Real-time monitoring of kitchen workload.
- A graphical dashboard showing order status and chef activity.

---

3. Users of the System

The system includes the following users:

- **Waiters:** Responsible for submitting customer orders into the system.
- **Chefs:** Responsible for retrieving and preparing orders from the queue.
- **Restaurant Manager:** Observes kitchen activity and order progress through the dashboard.

---

4. GUI Design

The system will include a **web-based graphical user interface** developed using:

- **HTML** for page structure
- **CSS** for styling and layout
- **PHP** for backend logic and dynamic data handling

The dashboard will display:

- Incoming customer orders
- Orders currently being prepared
- Completed orders
- Chef assignments
- Kitchen workload status

The interface will update dynamically to show real-time order processing activity.

---

5. Tools and Technologies

The project will be developed using the following tools and technologies:

- **Programming Language:** C++ / C (for simulation logic)
- **Frontend:** HTML, CSS
- **Backend:** PHP/ JAVASCRIPT
- **Concurrency Concepts:** Threads, Mutex, Semaphores
- **Data Structures:** Queue / Priority Queue

---

6. Expected Outcome

The final product will be a **Restaurant Kitchen Dashboard** that visually demonstrates how orders move through the system from submission to completion. The GUI will display incoming orders, chef assignments, and completed orders in real time.

The project will demonstrate how synchronization techniques such as mutexes and semaphores prevent conflicts when multiple threads interact with shared resources. It will also highlight the importance of efficient task scheduling and resource allocation in high-demand environments.

---

7. Conclusion

The Restaurant Order Management Simulator provides a practical demonstration of operating system concepts including concurrency, synchronization, and producer-consumer communication. By simulating a real-world restaurant environment, the system illustrates how multiple threads can efficiently coordinate tasks while maintaining data consistency and avoiding race conditions.